Assignment 3: Data Analysis and Mining

- Select a real-world dataset related to a specific domain (e.g., customer churn prediction, fraud detection).
- Implement a machine learning algorithm (e.g., decision tree, logistic regression) using Apache Mahout or a similar library.
  - Split the dataset into training, validation, and testing sets.
  - Evaluate the performance of the model using appropriate performance metrics (e.g.,
  - accuracy, precision, recall).

Assignment 3: Data Analysis and Mining
Detailed Practical Guidance
Title of the Practical
Data Analysis and Mining Using Machine Learning

1. Aim

To select a real-world dataset, perform data analysis and mining using a machine learning algorithm, divide the dataset into training, validation, and testing sets, and evaluate the model using appropriate performance metrics.

2. Objectives

After completing this practical, students will be able to:

1. Select and understand a real-world dataset.
2. Perform exploratory data analysis (EDA).
3. Preprocess and clean the dataset.
4. Implement a machine learning classification algorithm.
5. Divide data into training, validation, and testing sets.
6. Train a model using a suitable machine learning library.
7. Evaluate model performance using accuracy, precision, recall, and F1-score.
8. Interpret the results and identify important features.

3. Problem Statement

In this practical, a real-world dataset related to a specific domain such as customer churn prediction, fraud detection, or loan approval is selected.

A machine learning algorithm such as Decision Tree or Logistic Regression is implemented using Apache Mahout or a similar machine learning library.

The dataset is divided into:

- Training Dataset
- Validation Dataset
- Testing Dataset

The trained model is evaluated using appropriate performance metrics.

4. Recommended Dataset

For beginners, **Customer Churn Prediction Dataset** is highly recommended.

Dataset Domain:

Telecommunication / Customer Relationship Management

Objective:

Predict whether a customer will leave a telecom service.

Target Variable:

Variable	Meaning
Churn	Whether customer leaves the company
Yes	Customer has churned
No	Customer has not churned

Sample Features

Feature	Description
CustomerID	Unique customer identifier
Gender	Male/Female
SeniorCitizen	Whether customer is senior citizen
Tenure	Number of months customer stayed
MonthlyCharges	Monthly billing amount
TotalCharges	Total amount charged
Contract	Type of contract
PaymentMethod	Payment method
InternetService	Internet service type
Churn	Target variable

Suitable Dataset Sources

Students can download the Telco Customer Churn dataset from Kaggle or IBM sample datasets.

Recommended file name:

`WA_Fn-UseC_-Telco-Customer-Churn.csv`

5. Software and Hardware Requirements

Software Requirements

Software	Purpose
Python 3.x	Programming language
Jupyter Notebook / Google Colab	Execution environment
Pandas	Data manipulation
NumPy	Numerical computation
Matplotlib	Visualization
Seaborn	Statistical visualization
Scikit-learn	Machine learning implementation
Apache Mahout (Optional)	Distributed machine learning

Hardware Requirements

- Minimum 4 GB RAM
- Dual-core processor
- 10 GB free storage
- Internet connection for dataset download

6. Why Use Apache Mahout or Similar Library?

Apache Mahout is an open-source machine learning library designed for scalable data processing and machine learning.

It supports algorithms such as:

- Classification
- Clustering
- Recommendation
- Collaborative Filtering

For this practical, students may use:

Option 1: Apache Mahout

Suitable for Hadoop-based Big Data environments.

Option 2: Scikit-learn

Recommended for beginners because it is easier to install and execute.

Option 3: Apache Spark MLlib

Recommended for large datasets and distributed processing.

Practical Recommendation: Use Scikit-learn for implementation and explain how the same workflow can be extended to Apache Mahout or Spark MLlib.

7. Complete Practical Workflow

The overall workflow is:

Select Real-World Dataset
↓
Load Dataset
↓
Understand Dataset Structure
↓
Data Cleaning and Preprocessing
↓
Exploratory Data Analysis
↓
Feature Selection
↓
Split Dataset
(Training / Validation / Testing)
↓
Implement ML Algorithm
↓
Train Model
↓
Validate Model
↓
Test Final Model
↓
Evaluate Performance
↓
Interpret Results

8. Step 1: Import Required Libraries

</>Python

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder,
StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

#AfterRun
Run started
Initializing environment
Installing packages

Explanation

- `pandas`: Used for reading and manipulating datasets.
- `numpy`: Used for numerical operations.
- `matplotlib` and `seaborn`: Used for visualization.
- `train_test_split`: Used to divide the dataset.
- `LabelEncoder`: Converts categorical values into numerical values.
- `StandardScaler`: Standardizes numerical features.
- `DecisionTreeClassifier`: Machine learning classification algorithm.
- `LogisticRegression`: Another classification algorithm.
- `metrics`: Used to evaluate model performance.

9. Step 2: Load the Dataset

</>Python

```
df =
pd.read_csv("WA_Fn-UseC_-Telco-Customer-Chu
rn.csv")

print(df.head())
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'pd' is not defined

Run completed in 1351.699999988079ms

Output

The first five rows of the dataset will be displayed.

Example:

customerID	gender	tenure	MonthlyCharges	Churn
7590-VHVEG	Female	1	29.85	No
5575-GNVDE	Male	34	56.95	No
3668-QPYBK	Male	2	53.85	Yes

10. Step 3: Understand Dataset Structure

</>Python

```
print(df.shape)
print(df.columns)
print(df.info())
print(df.describe())
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1123.3999999761581ms

Important Functions

Function	Purpose
`shape`	Shows number of rows and columns
`columns`	Displays column names
`info()`	Shows data types and missing values
`describe()`	Provides statistical summary

Example Interpretation

If output is:

```
Rows: 7043
Columns: 21
```

It means the dataset contains 7043 customer records and 21 attributes.

11. Step 4: Check Missing Values

</>Python

```
print(df.isnull().sum())
```

Explanation

Missing values may affect machine learning performance.

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 947.8999999761581ms

Explanation

Missing values may affect machine learning performance.

For example:

Column	Missing Values
TotalCharges	11
Tenure	0
MonthlyCharges	0

Handling Missing Values

</>Python

```
df['TotalCharges'] = pd.to_numeric(
    df['TotalCharges'],
    errors='coerce'
)

df['TotalCharges'].fillna(
    df['TotalCharges'].median(),
    inplace=True
)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'pd' is not defined

Run completed in 1060ms

Explanation

- Converts TotalCharges into numeric format.
- Invalid values are converted to missing values (`NaN`).
- Missing values are replaced with median.

12. Step 5: Remove Unnecessary Columns

Customer ID does not contribute significantly to prediction.

</>Python

```
df.drop('customerID', axis=1, inplace=True)
```

Why Remove CustomerID?

CustomerID is a unique identifier, not a meaningful predictive feature.

Including it may increase complexity without improving prediction.

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1005.6999999880791ms

Why Remove CustomerID?

CustomerID is a unique identifier, not a meaningful predictive feature.

Including it may increase complexity without improving prediction.

13. Step 6: Convert Categorical Data into Numerical Data

Machine learning algorithms generally require numerical input.

Example categorical values:

These must be converted into numerical values.

Gender
Male
Female

</>Python

```
df = pd.get_dummies(df, drop_first=True)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'pd' is not defined

Run completed in 1069.800000011921ms

Explanation

One-hot encoding converts categorical variables into binary numerical columns.

Example:

Gender	Gender_Male
Female	0
Male	1

14. Step 7: Separate Features and Target Variable

</>Python

```
X = df.drop('Churn_Yes', axis=1)
y = df['Churn_Yes']
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1021.7000000476837ms

Explanation

- `X`: Independent variables/features.
- `y`: Dependent variable/target.

Example

```
Features (X):
Tenure, MonthlyCharges, Contract,
PaymentMethod, etc.

Target (y):
Churn
```

15. Step 8: Split Dataset into Training, Validation, and Testing Sets

This is an important requirement of the assignment.

Recommended Dataset Distribution

Dataset	Percentage	Purpose
Training Set	70%	Train the machine learning model
Validation Set	15%	Tune parameters and compare models
Testing Set	15%	Final unbiased evaluation

Python Implementation

First, split into training and temporary data:

</>Python

```
X_train, X_temp, y_train, y_temp =
train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'train_test_split' is not defined

Run completed in 1017.3000000119209ms

Now divide temporary data equally into validation and testing:

```
</>Python
X_val, X_test, y_val, y_test = train_test_split(    X_temp,
y_temp,    test_size=0.50,    random_state=42,
stratify=y_temp)
```

```
#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'train_test_split' is not defined
Run completed in 1036.6000000238419ms
```

Verify Dataset Sizes

</>Python

```
print("Training Data:", X_train.shape)
print("Validation Data:", X_val.shape)
print("Testing Data:", X_test.shape)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'X_train' is not defined

Run completed in 1009.6999999880791ms

Expected Distribution

Approximately:

```
Training Set: 70%
Validation Set: 15%
Testing Set: 15%
```

16. Why Three Different Datasets?

Training Dataset

Used to teach the machine learning algorithm patterns in the data.

Validation Dataset

Used to:

- Tune hyperparameters.
- Compare different algorithms.
- Prevent overfitting.
- Select the best model.

Testing Dataset

Used only at the end to measure final model performance on unseen data.

17. Step 9: Feature Scaling

Some machine learning algorithms perform better when numerical features are standardized.

</>Python

```
scaler = StandardScaler()

X_train_scaled =
scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'StandardScaler' is not defined

Run completed in 1065.800000011921ms

Important Note

The scaler must be fitted only on training data.

This prevents data leakage.

18. Step 10: Implement Machine Learning Algorithm

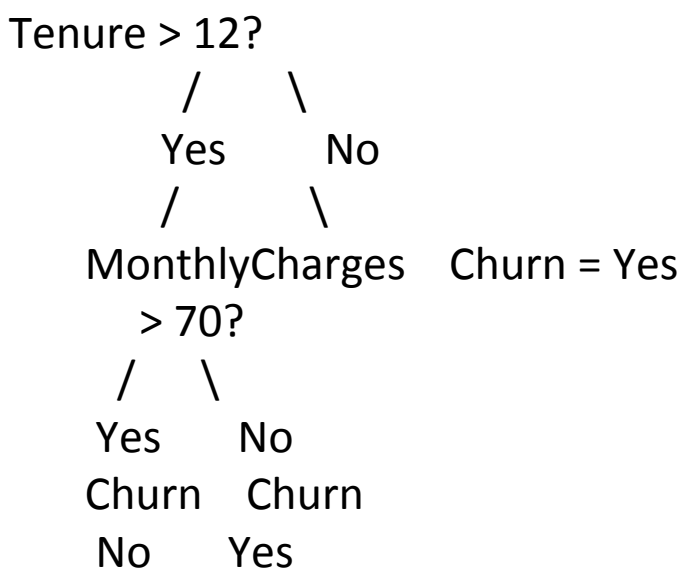
Algorithm 1: Decision Tree Classifier

A Decision Tree is a supervised machine learning algorithm used for classification and regression.

Working Principle

A decision tree makes decisions by repeatedly splitting data based on feature values.

Example:



Advantages

- Easy to understand.
- Works with numerical and categorical data.
- Does not require extensive data scaling.
- Useful for feature importance analysis.

Python Code
</>Python

```
model = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=5,
    random_state=42
)

model.fit(X_train, y_train)
```

#AfterRun
Run
Run started
Initializing environment
Installing packages
Running code
NameError: name 'DecisionTreeClassifier' is not defined
Run completed in 1052.5ms

19. Step 11: Validate the Model

</>Python

```
y_val_pred = model.predict(X_val)

val_accuracy = accuracy_score(y_val,
y_val_pred)
val_precision = precision_score(y_val,
y_val_pred)
val_recall = recall_score(y_val,
y_val_pred)
val_f1 = f1_score(y_val, y_val_pred)

print("Validation Accuracy:", val_accuracy)
print("Validation Precision:",
val_precision)
print("Validation Recall:", val_recall)
print("Validation F1 Score:", val_f1)
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'model' is not defined
Run completed in 1057ms

20. Hyperparameter Tuning

Hyperparameters control the behavior of a machine learning algorithm.

For Decision Tree:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`
- `criterion`

Experiment with Different Tree Depths

</>Python

```
depths = [3, 5, 7, 10, None]

for depth in depths:

    dt_model = DecisionTreeClassifier(
        max_depth=depth,
        random_state=42
    )

    dt_model.fit(X_train, y_train)

    y_val_pred = dt_model.predict(X_val)

    accuracy = accuracy_score(y_val,
y_val_pred)

    print("Depth:", depth, "Validation
Accuracy:", accuracy)
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'DecisionTreeClassifier' is not defined
Run completed in 1029.3999999761581ms

Select Best Model

The model with the best validation performance is selected for final testing.

21. Algorithm 2: Logistic Regression

Logistic Regression is a supervised machine learning algorithm used primarily for binary classification.

Example Applications

- Customer churn prediction.
- Disease detection.
- Fraud detection.
- Loan approval prediction.

Working Principle

Logistic Regression calculates the probability of an event using the sigmoid function.

Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The output ranges between 0 and 1.

Interpretation

Probability	Prediction
< 0.5	Class 0
≥ 0.5	Class 1

22. Python Implementation of Logistic Regression

</>Python

```
log_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

log_model.fit(X_train_scaled, y_train)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'LogisticRegression' is not defined

Run completed in 1046.19999988079ms

Validation

</>Python

```
y_val_pred_log =
log_model.predict(X_val_scaled)

print("Validation Accuracy:",
      accuracy_score(y_val,
y_val_pred_log))

print("Validation Precision:",
      precision_score(y_val,
y_val_pred_log))

print("Validation Recall:",
      recall_score(y_val, y_val_pred_log))

print("Validation F1 Score:",
      f1_score(y_val, y_val_pred_log))
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'log_model' is not defined
Run completed in 1093.6000000238419ms

23. Step 12: Final Model Testing

After selecting the best model using validation data, evaluate it on the testing dataset.

</>Python

```
final_model = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=5,
    random_state=42
)

final_model.fit(X_train, y_train)

y_test_pred = final_model.predict(X_test)
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'DecisionTreeClassifier' is not defined
Run completed in 1006.2999999523163ms

24. Performance Evaluation Metrics

The following metrics are used to evaluate classification models.

24.1 Accuracy

Accuracy measures the percentage of correctly classified observations.

Formula

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

Example

If 90 out of 100 predictions are correct:

Accuracy=100 / 90=90%

Python
</>Python

```
accuracy = accuracy_score(y_test,
y_test_pred)

print("Accuracy:", accuracy)
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'accuracy_score' is not defined
Run completed in 1091.8999999761581ms

25. Precision

Precision measures how many predicted positive cases are actually positive.

Formula

$$Precision = \frac{TP}{TP + FP}$$

Example

If the model predicts 20 customers will churn, but only 15 actually churn:

$$Precision = \frac{15}{20} = 75\%$$

Importance

Precision is important when false positives are costly.

For example, incorrectly identifying a loyal customer as likely to leave may result in unnecessary retention offers.

26. Recall

Recall measures how many actual positive cases are correctly identified.

Formula

$$Recall = \frac{TP}{TP + FN}$$

Example

If 30 customers actually churn and the model correctly identifies 24:

$$Recall = \frac{24}{30} = 80\%$$

Importance

Recall is especially important in:

- Fraud detection.
- Medical diagnosis.
- Security applications.

27. F1-Score

F1-score is the harmonic mean of Precision and Recall.

Formula

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

F1-score is useful when the dataset is imbalanced.

28. Confusion Matrix

A confusion matrix summarizes classification predictions.

Actual / Predicted	No Churn	Churn
No Churn	TN	FP
Churn	FN	TP

Python Code

</>Python

```
cm = confusion_matrix(y_test, y_test_pred)

print(cm)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'confusion_matrix' is not defined

Run completed in 1065.300000011921ms

Visualization

</>Python

```
plt.figure(figsize=(6,4))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'plt' is not defined
Run completed in 1033.7000000476837ms

29. Complete Performance Evaluation Code

</>Python

```
accuracy = accuracy_score(y_test,
y_test_pred)
precision = precision_score(y_test,
y_test_pred)
recall = recall_score(y_test, y_test_pred)
f1 = f1_score(y_test, y_test_pred)

print("Model Performance")
print("------------------")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)

print("\nClassification Report:")
print(classification_report(y_test,
y_test_pred))
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'accuracy_score' is not defined
Run completed in 1024.5ms

30. Sample Output

Actual results depend on dataset preprocessing and algorithm parameters.

Example:

```
Model Performance
------------------
Accuracy : 0.78
Precision: 0.65
Recall   : 0.58
F1 Score : 0.61
```

Interpretation

Metric	Value	Interpretation
Accuracy	78%	78% predictions are correct
Precision	65%	65% predicted churn cases are correct
Recall	58%	Model identifies 58% of actual churn customers
F1 Score	61%	Balance between precision and recall

31. Comparing Two Machine Learning Algorithms

Students can compare Decision Tree and Logistic Regression.

Metric	Decision Tree	Logistic Regression
Accuracy	78%	80%
Precision	65%	68%
Recall	58%	62%
F1 Score	61%	65%

Conclusion from Comparison

The algorithm with higher validation and testing performance may be selected as the better model.

However, accuracy alone should not determine the best algorithm.

Precision, Recall, and F1-score should also be considered.

32. Data Visualization for Analysis

Churn Distribution

</>Python

```
sns.countplot(x='Churn', data=pd.read_csv(
    "WA_Fn-UseC_-Telco-Customer-Churn.csv"
))

plt.title("Customer Churn Distribution")
plt.show()
```

Purpose

Shows the number of customers who churned versus those who stayed.

```
#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'sns' is not defined
Run completed in 1039.899999761581ms
```

Monthly Charges Distribution
</>Python

```
plt.figure(figsize=(8,5))

sns.histplot(df['MonthlyCharges'],
kde=True)

plt.title("Distribution of Monthly
Charges")
plt.show()
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'plt' is not defined
Run completed in 1085.800000011921ms

Tenure vs Churn

</>Python

```
sns.boxplot(
    x='Churn',
    y='tenure',
    data=pd.read_csv(

"WA_Fn-UseC_-Telco-Customer-Churn.csv"
    )
)

plt.title("Tenure Distribution by Churn")
plt.show()
```

Interpretation

Customers with shorter tenure may have a higher probability of churn.

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'sns' is not defined
Run completed in 1006ms

33. Feature Importance in Decision Tree

Decision Trees provide feature importance values.

</>Python

```
importance =
final_model.feature_importances_

feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importance
})

feature_importance =
feature_importance.sort_values(
    by='Importance',
    ascending=False
)

print(feature_importance)
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'final_model' is not defined
Run completed in 1038.5ms

Visualization

</>Python

```
plt.figure(figsize=(10,6))

sns.barplot(
    x='Importance',
    y='Feature',
    data=feature_importance.head(10)
)

plt.title("Top Important Features")
plt.show()
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'plt' is not defined
Run completed in 1046ms

Possible Important Features

- Tenure
- Contract Type
- Monthly Charges
- Internet Service
- Total Charges

34. Apache Mahout Implementation Concept

Apache Mahout is suitable for large-scale machine learning applications.

General Architecture

```
Raw Dataset
     ↓
HDFS Storage
     ↓
Data Preprocessing
     ↓
Feature Engineering
     ↓
Apache Mahout Algorithm
     ↓
Model Training
     ↓
Model Evaluation
```

Mahout Algorithms

Algorithm	Application
Logistic Regression	Binary Classification
Decision Forest	Classification
Naive Bayes	Text Classification
K-Means	Clustering
Collaborative Filtering	Recommendation Systems

Note for Students

Apache Mahout traditionally supports distributed machine learning on Hadoop ecosystems. In modern practical environments, Scikit-learn or Spark MLlib is often easier to implement, while Mahout can be discussed as the Big Data machine learning framework.

35. Optional Apache Spark MLlib Approach

For Big Data Analytics students, Apache Spark MLlib is also highly relevant.

Spark Machine Learning Workflow

```
Dataset
   ↓
Spark DataFrame
   ↓
Data Cleaning
   ↓
VectorAssembler
   ↓
Train/Test Split
   ↓
Decision Tree Classifier
   ↓
Predictions
   ↓
MulticlassClassificationEvaluator
```

Example Spark Code

</>Python

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("CustomerChurnPrediction") \
    .getOrCreate()

df = spark.read.csv(
    "WA_Fn-UseC_-Telco-Customer-Churn.csv",
    header=True,
    inferSchema=True
)

df.show(5)
```

For beginners, Scikit-learn implementation is recommended for the actual execution.

#AfterRun
Run started
Initializing environment
Installing packages
Running code
ModuleNotFoundError: No module named 'pyspark'
Run completed in 3187.2000000476837ms

36. Important Concepts Covered in This Practical

Concept	Description
Data Mining	Extracting useful patterns from data
Machine Learning	Learning patterns automatically from data
Classification	Predicting categories/classes
Training Set	Used to train the model
Validation Set	Used for model selection
Testing Set	Used for final evaluation
Overfitting	Model performs well on training data but poorly on unseen data
Underfitting	Model is too simple to learn patterns
Accuracy	Overall correctness
Precision	Correctness of positive predictions
Recall	Ability to identify actual positive cases
F1 Score	Balance between precision and recall

37. Advantages of This Practical

1. Demonstrates real-world application of machine learning.
2. Provides hands-on experience with data preprocessing.
3. Introduces classification algorithms.
4. Teaches proper dataset splitting techniques.
5. Helps students understand model evaluation.
6. Connects Big Data Analytics with machine learning.
7. Develops practical skills in Python and machine learning libraries.

38. Expected Learning Outcome

After completing this assignment, students should be able to:

- Select a real-world dataset.
- Analyze dataset characteristics.
- Handle missing and categorical data.
- Apply machine learning algorithms.
- Split data into training, validation, and testing sets.
- Calculate classification performance metrics.
- Compare different machine learning models.
- Interpret and report model results.

39. Viva Questions

Basic Questions

1. What is data mining?
2. What is the difference between data analysis and data mining?
3. What is machine learning?
4. What is supervised learning?
5. What is classification?
6. Why is customer churn prediction important?

Dataset Questions

7. Why was this dataset selected?
8. What is the target variable?
9. What are independent and dependent variables?
10. Why should missing values be handled?
11. Why is CustomerID removed?

Model Questions

12. What is a Decision Tree?
13. What is Logistic Regression?
14. What is overfitting?
15. What is hyperparameter tuning?
16. Why is validation data required?

Evaluation Questions

17. What is accuracy?
18. What is precision?
19. What is recall?
20. What is F1-score?
21. What is a confusion matrix?
22. When is recall more important than precision?

40. Final Practical Conclusion

In this practical, a real-world customer churn dataset was analyzed and preprocessed for machine learning. The dataset was divided into training, validation, and testing subsets. Classification algorithms such as Decision Tree and Logistic Regression were implemented using Python and Scikit-learn. The performance of the models was evaluated using accuracy, precision, recall, F1-score, and confusion matrix.

The practical demonstrates how machine learning and data mining techniques can be applied to solve real-world business problems and make data-driven decisions.

Suggested Practical Report Format

Students can submit their assignment in the following format:

1. Title
2. Aim
3. Objectives
4. Problem Statement
5. Dataset Description
6. Software and Hardware Requirements
7. Theory
8. Algorithm Used
9. Implementation / Source Code
10. Dataset Splitting
11. Model Training
12. Validation Results
13. Testing Results
14. Performance Metrics
15. Graphs and Visualizations
16. Result and Discussion
17. Conclusion
18. Viva Questions

Recommended Implementation Choice

For your Big Data Analytics use **Customer Churn Prediction + Decision Tree Classifier + Python Scikit-learn**, because it is easy to execute, demonstrates the complete machine learning pipeline, and can later be extended to Apache Spark MLlib or Apache Mahout for large-scale processing.